PREBEL S.A.S BIC

Area de automatización y desarrollo

Prebel
BEAUTYCARE PARTNERS

Manual de Integración con Microsoft Graph y SharePoint

Registro de aplicación, permisos, configuración del sitio
y conexión de la aplicación para persistencia de archivos

Guía paso a paso para replicar la configuración de Microsoft Entra ID y SharePoint Online necesaria para que una aplicación externa almacene archivos y metadatos mediante Microsoft Graph API.

Medellín, Colombia · 2026

Presentación

Este manual documenta el proceso completo para conectar una aplicación externa con una biblioteca de documentos y una lista de SharePoint Online, utilizando Microsoft Graph API como capa de integración. El objetivo es que cualquier persona del equipo pueda replicar esta configuración sobre un sitio nuevo, sin depender de quien la ejecutó por primera vez.

El proceso se dividió en tres bloques: el registro y configuración de la aplicación en Microsoft Entra ID, la preparación del sitio de SharePoint destino, y la validación de la conexión mediante Postman antes de integrarla en el código de la aplicación.

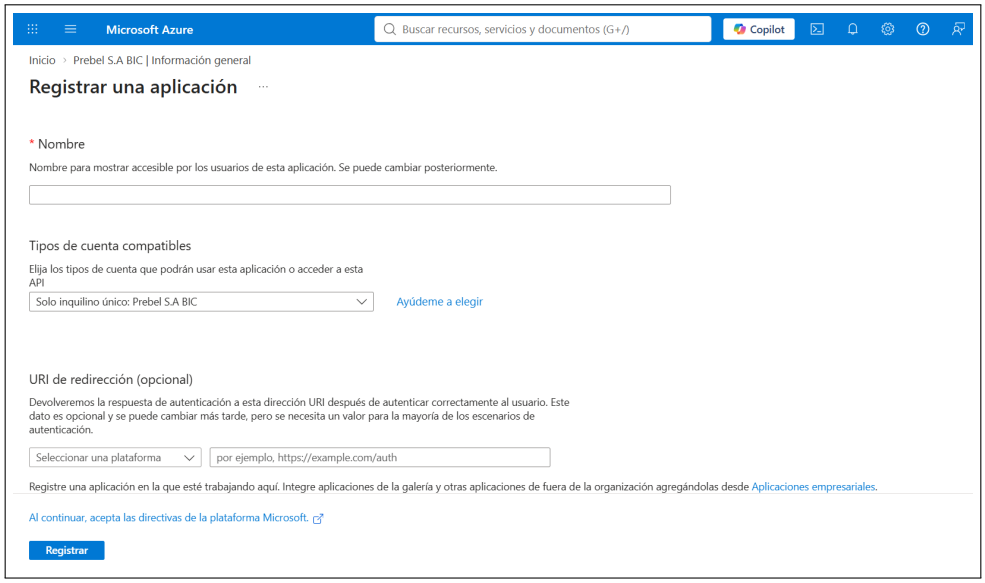
1 Registro de la aplicación en Microsoft Entra ID

1. Registrar la aplicación

Dónde: portal.azure.com / entra.microsoft.com → App registrations → New registration

Se registra una nueva aplicación en el tenant de la organización. Este registro genera dos identificadores que se usarán en toda la integración:

- **Application (client) ID**: identifica la aplicación.
- **Directory (tenant) ID**: identifica la organización (Prebel).

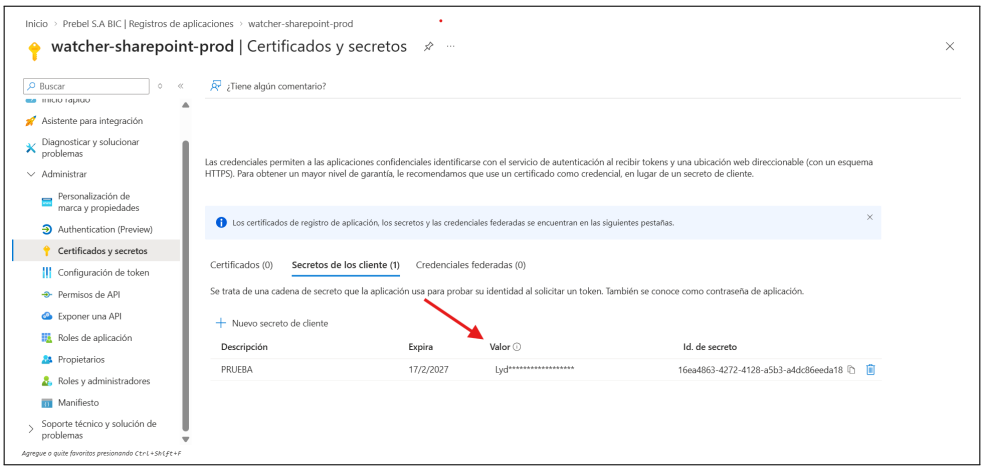


2. Generar el Client Secret

Dónde: La aplicación registrada → Certificates & secrets → New client secret

El client secret funciona como la contraseña de la aplicación. Al crearlo, el portal muestra dos columnas: Value y Secret ID. Únicamente el Value es válido para autenticar la aplicación.

> **Importante**
>
> El Value solo se muestra una vez, en el momento de crear el secreto. Si se recarga la página o se navega fuera antes de copiarlo, no se puede recuperar y es necesario generar uno nuevo.



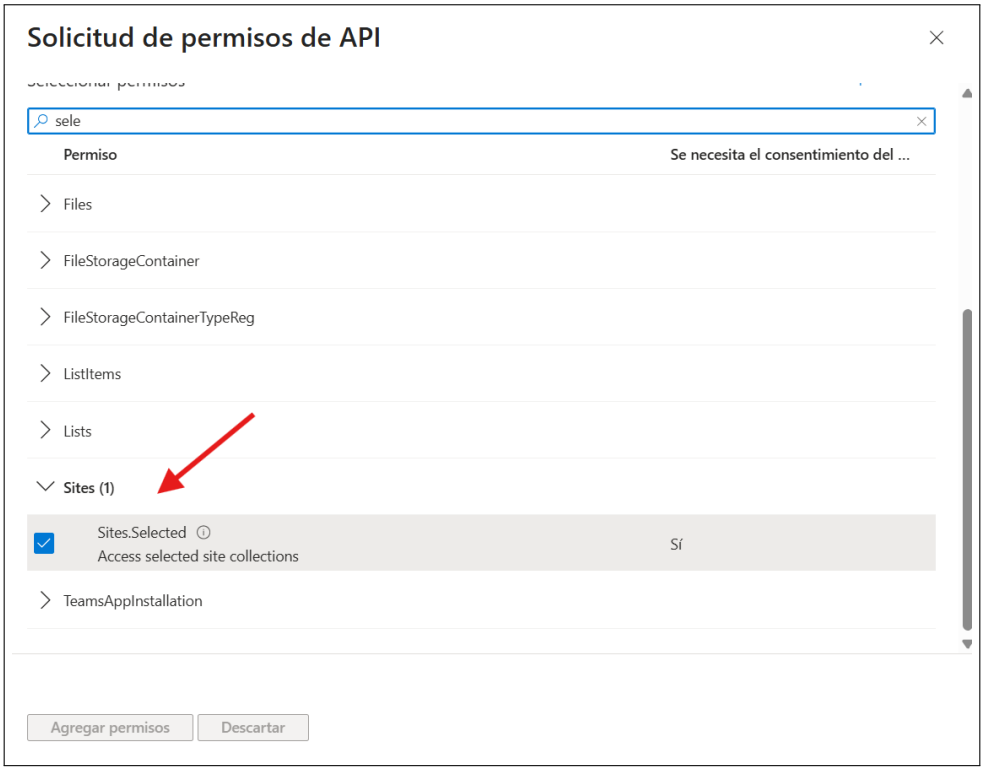
3. Configurar el permiso Sites.Selected

Dónde: La aplicación registrada → API permissions → Add a permission → Microsoft Graph

Se otorga el permiso `Sites.Selected`, eligiendo explícitamente el tipo Application permissions (no *Delegated permissions*). Esto es lo que le permite a la aplicación actuar de forma autónoma, sin que haya un usuario con sesión iniciada de por medio.

Importante

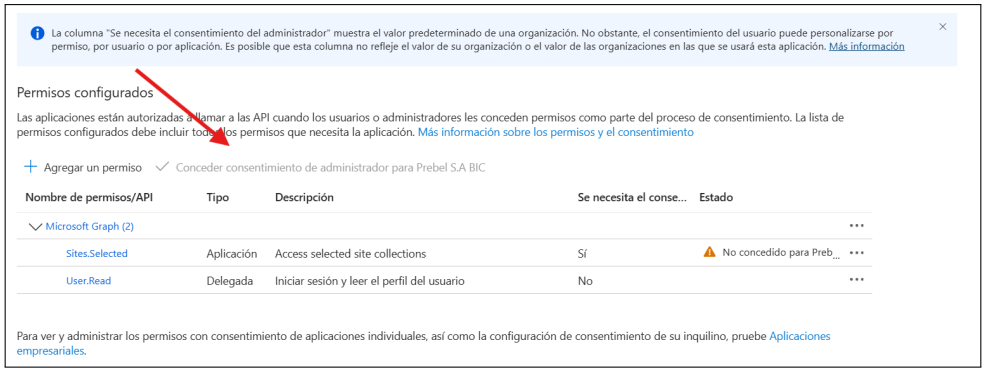
Si el permiso se agrega como *Delegated* en lugar de *Application*, la aplicación sí logra obtener un token, pero cualquier llamada a Microsoft Graph devuelve error 401 No autorizado.



4. Otorgar el consentimiento del administrador

Dónde: La aplicación registrada → API permissions → botón *Grant admin consent*

Un administrador del tenant debe aprobar el permiso configurado en el paso anterior. Sin este consentimiento, el permiso queda listado pero inactivo.

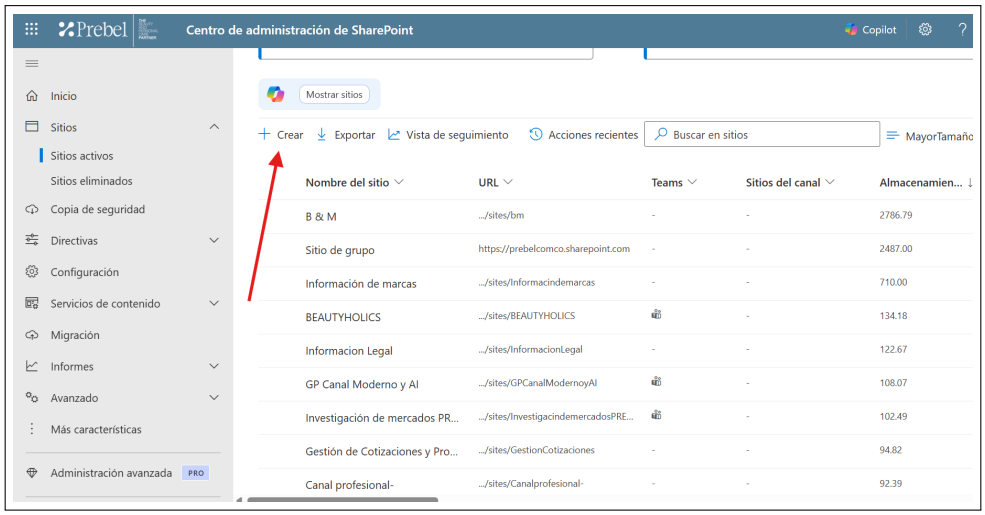


2 Preparación del sitio de SharePoint

5. Crear el sitio de SharePoint

Dónde: SharePoint (portal web) → botón + Crear sitio → Sitio de equipo

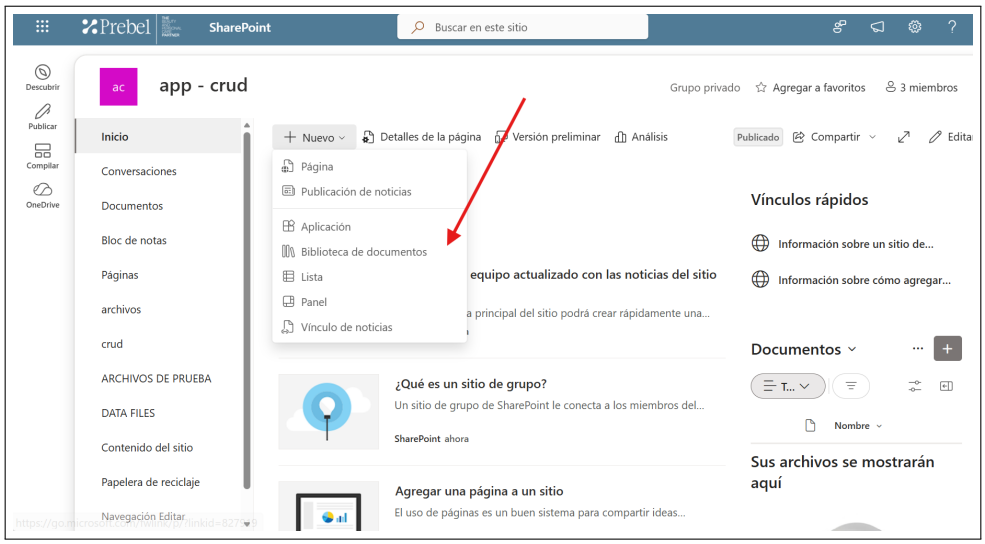
Se crea el sitio que va a alojar la biblioteca de archivos y la lista de metadatos. El nombre elegido define la ruta final del sitio (por ejemplo, un sitio llamado `app-crud` queda disponible en `https://{tenant}.sharepoint.com/sites/app-crud`).



6. Crear la biblioteca de documentos

Dónde: Dentro del sitio → + Nuevo → Biblioteca de documentos

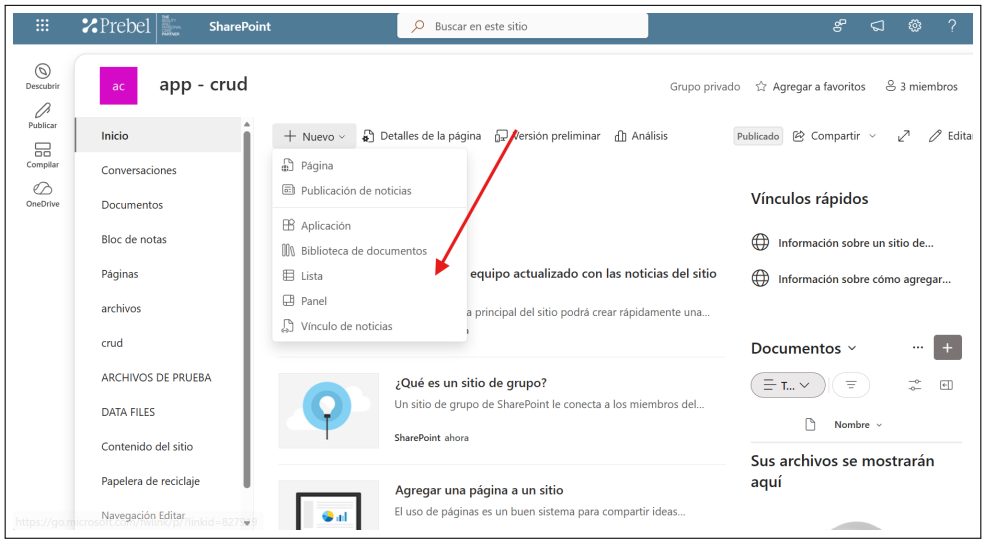
Se crea una biblioteca en blanco, destinada exclusivamente a alojar los archivos que la aplicación va a subir mediante Microsoft Graph.



7. Crear la lista de metadatos

Dónde: Dentro del sitio → + Nuevo → Lista → Lista en blanco

Se crea una lista independiente de la biblioteca, destinada a almacenar la metadata asociada a cada archivo subido (nombre, fecha, usuario, referencia al archivo, entre otros campos según necesidad).



8. Autorizar la aplicación sobre el sitio específico

Dónde: Postman — solicitud POST autenticada con una cuenta administradora del tenant

El permiso `Sites.Selected` habilita el tipo de acceso a nivel de Microsoft Graph, pero no otorga acceso a ningún sitio por sí solo. Es necesario un paso adicional: autorizar explícitamente a la aplicación sobre el sitio creado, indicando el rol que va a tener (lectura o escritura). Esta solicitud se envía una única vez por cada sitio nuevo que la aplicación deba usar:

```
POST https://graph.microsoft.com/v1.0/sites/{hostname}:/sites/{site_path
    }:/permissions
```

```
Cuerpo (JSON):
{
  "roles": ["write"],
  "grantedToIdentities": [
    {
      "application": {
        "id": "{client_id}",
        "displayName": "{nombre_de_la_app}"
      }
    }
  ]
}
```

Importante

Sin este paso, cualquier solicitud posterior de la aplicación sobre el sitio devuelve error 403 Prohibido, aunque el permiso de Microsoft Graph ya esté aprobado.

3 Validación de la conexión con Postman

Antes de integrar la conexión en el código de la aplicación, se valida cada llamada de forma manual en Postman. Esto permite confirmar que las credenciales, permisos y configuración del sitio son correctos antes de depender del código.

9. Obtener el token de acceso

Dónde: Postman — solicitud *Obtener Token*

Se solicita el token de acceso mediante el flujo *client credentials*, usando el `tenant_id`, `client_id` y `client_secret` obtenidos en los pasos 1 y 2.

```
POST https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token

Cuerpo (x-www-form-urlencoded):
grant_type=client_credentials
client_id={client_id}
client_secret={client_secret}
scope=https://graph.microsoft.com/.default
```

La respuesta incluye el `access_token`, válido durante aproximadamente una hora.

10. Obtener el identificador del sitio (site_id)

Dónde: Postman — solicitud *Obtener Site ID*, con el token del paso anterior

```
GET https://graph.microsoft.com/v1.0/sites/{hostname}:/sites/{site_path}
Authorization: Bearer {access_token}
```

La respuesta incluye el campo `id`, un valor compuesto (hostname + dos identificadores) que se usará en las siguientes solicitudes.

11. Obtener el identificador de la biblioteca (drive_id)

Dónde: Postman — solicitud GET sobre /drives

```
GET https://graph.microsoft.com/v1.0/sites/{site_id}/drives
Authorization: Bearer {access_token}
```

La respuesta lista todas las bibliotecas del sitio. Se identifica la biblioteca creada en el paso 6 por su campo `name` y se copia su `id`.

Importante

El nombre por defecto de la biblioteca principal depende del idioma del sitio (*Documents* en inglés, *Documentos* en español). Fijar el `drive_id` directamente evita que la aplicación falle por esta diferencia de nombre.

12. Confirmar el identificador de la lista de metadatos

Dónde: Postman — solicitud *Buscar listas*

```
GET https://graph.microsoft.com/v1.0/sites/{site_id}/lists
Authorization: Bearer {access_token}
```

Se identifica la lista creada en el paso 7 por su campo `name` y se copia su `id`.

13. Probar la inserción de un registro de prueba

Dónde: Postman — solicitud *TestInsert*

Se valida que la aplicación puede escribir en la lista de metadatos, no solo leerla:

```
POST https://graph.microsoft.com/v1.0/sites/{site_id}/lists/{list_id}/
    items
Authorization: Bearer {access_token}
Content-Type: application/json

{
  "fields": {
    "Title": "Prueba de conexion"
  }
}
```

14. Probar la subida de un archivo

Dónde: Postman — solicitud *Subir Imagen*

Se valida la subida de un archivo binario a la biblioteca, usando el `drive_id` obtenido en el paso 11:

```
PUT https://graph.microsoft.com/v1.0/drives/{drive_id}/root:/prueba.txt:/
    content
Authorization: Bearer {access_token}
Content-Type: text/plain

(contenido del archivo en el cuerpo de la solicitud)
```

Si la solicitud es exitosa, el archivo aparece de inmediato en la biblioteca dentro del sitio de SharePoint.

4 Configuración de la aplicación

Una vez validada la conexión en Postman, los mismos valores se trasladan a la configuración de la aplicación, para que esta realice las llamadas directamente, sin depender de Postman.

15. Definir las variables de entorno

Dónde: Archivo de configuración (`.env`) de la aplicación

Estas son las variables tal como las usa el template funcional (*Graph Upload PoC*), verificado end-to-end contra el sitio `app-crud`:

```
# Registro de aplicacion en Entra ID
TENANT_ID=
CLIENT_ID=
CLIENT_SECRET=

# Sitio de SharePoint destino
SHAREPOINT_HOSTNAME=
SHAREPOINT_SITE_PATH=
SHAREPOINT_SITE_ID=

# Biblioteca de archivos (recomendado: ID directo)
SHAREPOINT_DRIVE_ID=
SHAREPOINT_LIBRARY=

# Carpeta destino dentro de la biblioteca (opcional, vacio = raiz)
TARGET_FOLDER=

# Lista de metadatos (vacio = sube sin catalogar)
SHAREPOINT_METADATA_LIST_ID=

# Nombres internos de columna de la lista (ver seccion 4)
LIST_COL_DRIVE_ITEM_ID=DriveItemId
LIST_COL_SIZE=TamanoBytes
LIST_COL_MIME=TipoMime
LIST_COL_URL=UrlArchivo
```

Importante

El `client_secret` nunca se documenta ni se comparte en texto plano fuera del gestor de configuración de la aplicación (archivo `.env` o gestor de secretos). Este manual solo referencia el nombre de la variable, no su valor.

Resolución directa vs. por nombre

El código acepta `SHAREPOINT_SITE_ID` y `SHAREPOINT_DRIVE_ID` directos, y solo si están vacíos los resuelve por hostname/ruta y por nombre de biblioteca (`SHAREPOINT_LIBRARY`). Usar los ID directos ahorra llamadas y elimina el problema del nombre de biblioteca por defecto, que varía según el idioma del sitio (*Documents* vs *Documentos*).

5 El template funcional: Graph Upload PoC

Todo lo anterior se validó y quedó implementado en un template de referencia, con arquitectura hexagonal, que otros equipos de Prebel pueden adoptar directamente. Esta sección documenta su funcionamiento interno, para que quien lo reciba entienda las decisiones de diseño y no repita los mismos hallazgos.

5.1 Qué hace, a nivel funcional

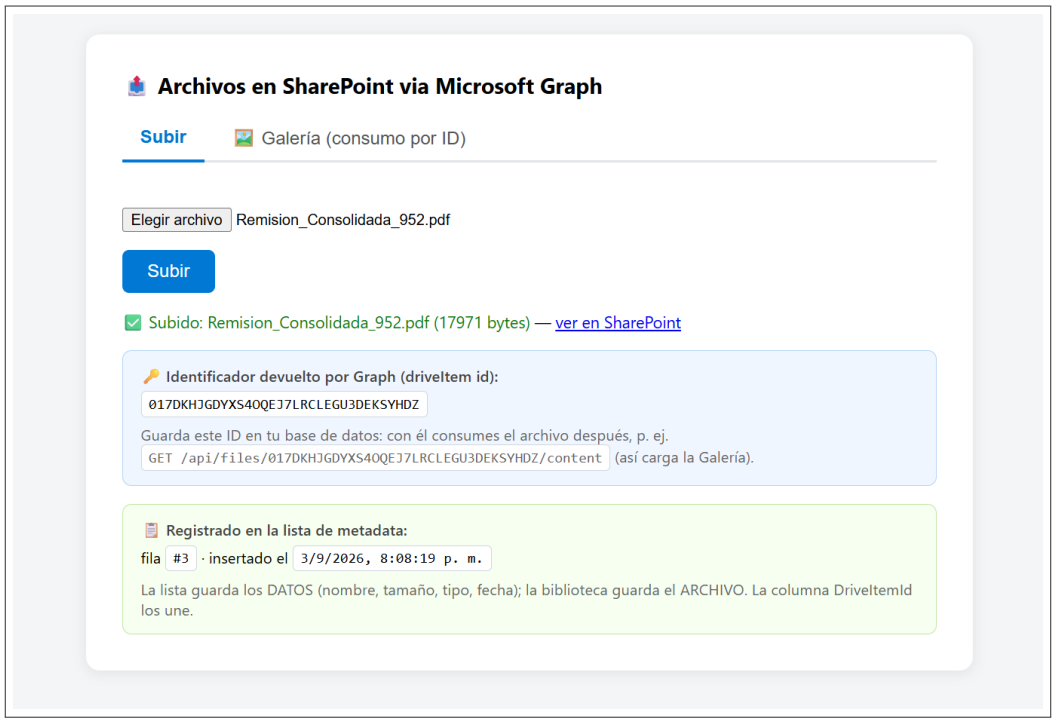
Un usuario abre una página web, selecciona un archivo y lo sube. El backend lo guarda en la biblioteca de documentos de SharePoint y, en paralelo, registra sus datos en la lista de metadatos que funciona como catálogo. La página tiene una segunda pestaña, Galería, que muestra los archivos ya guardados, recuperándolos por su identificador.

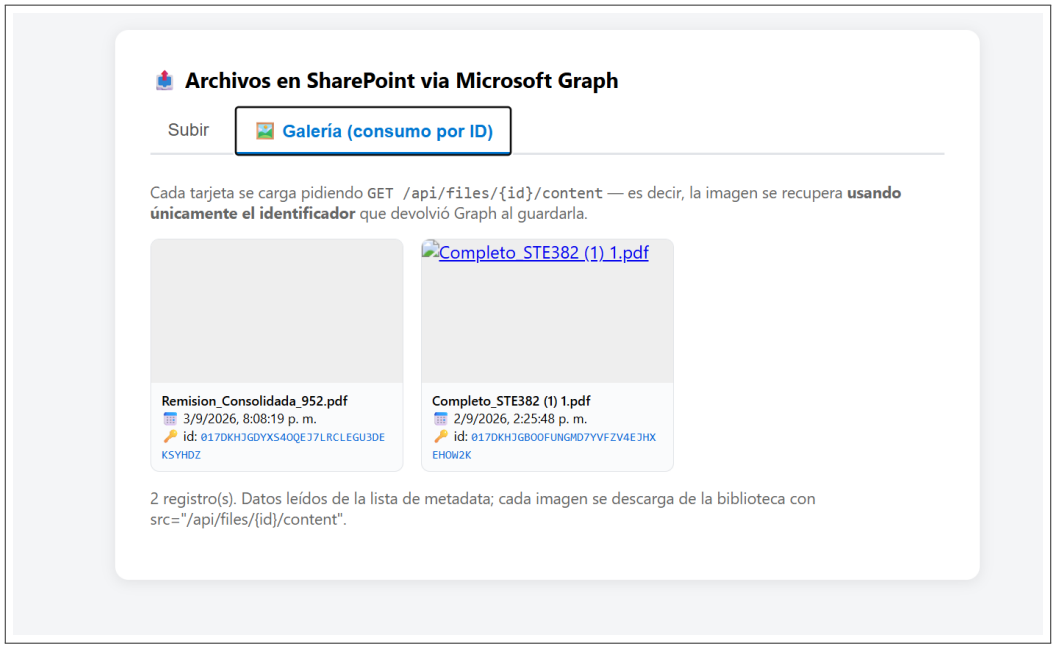
La separación entre ambas piezas es la decisión central del diseño: la biblioteca guarda el archivo binario, la lista guarda los datos sobre ese archivo, y la columna DriveItemId de la lista contiene el identificador que Graph le asignó al archivo en la biblioteca — es la llave que une ambos mundos. En un aplicativo real, esa lista sería reemplazada por la base de datos propia del proyecto.

El punto didáctico del template es el ciclo de vida de ese identificador: al subir un archivo, Graph devuelve un `driveItem id`, y ese id es lo único que se necesita para recuperar el archivo después. La interfaz lo muestra tras cada subida, y la galería lo usa directamente como `src` de cada imagen, para dejar explícito que el consumo posterior se hace por identificador, no por ruta ni por nombre.

Interfaz del template

El frontend de demostración (`public/index.html`) tiene dos pestañas:



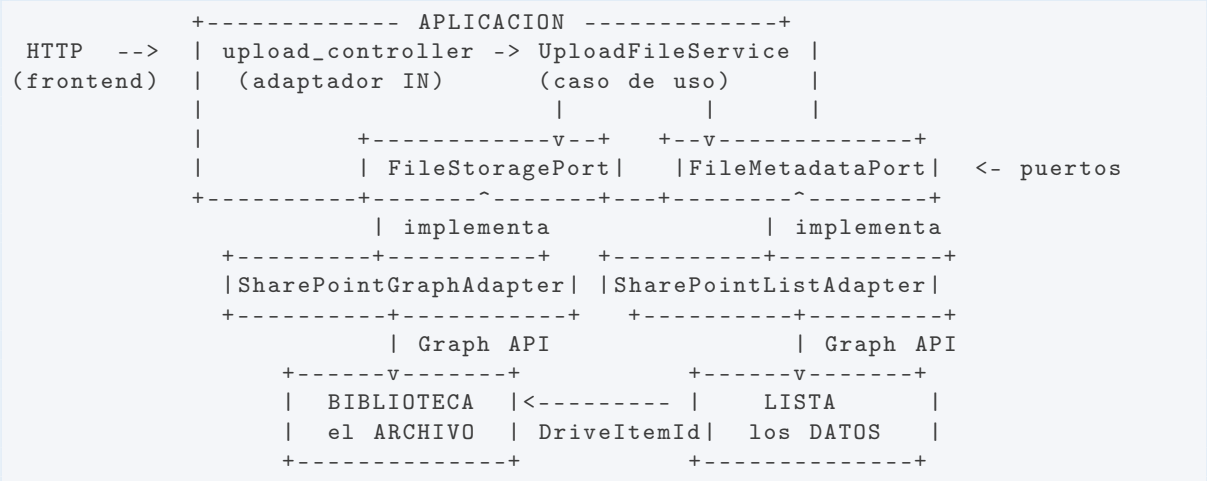


5.2 Flujo de una subida

1. El navegador envía el archivo como `multipart/form-data` a `POST /api/files`.
2. El controlador lo convierte en un objeto de dominio y llama al caso de uso.
3. Se valida: nombre no vacío, archivo no vacío, tamaño bajo el límite (250 MB).
4. El archivo sube a la biblioteca: `PUT /drives/{drive-id}/root:/{ruta}:/content`.
5. Se inserta la fila de metadata: `POST /sites/{site-id}/lists/{list-id}/items`.
6. La respuesta devuelve el `id` del archivo y el registro con la fecha de inserción.

5.3 Arquitectura

Arquitectura hexagonal (puertos y adaptadores): las dependencias apuntan hacia adentro. El dominio no depende de nada; la aplicación define interfaces (puertos) según sus necesidades; la infraestructura las implementa (adaptadores).



El motivo de tener dos puertos es el punto instructivo del template: un caso de uso puede orquestar varias dependencias sin acoplarse a ninguna. Al llevar esto a producción, lo esperable es reemplazar `SharePointListAdapter` por un adaptador contra la base de datos propia del proyecto, sin que el

caso de uso ni el dominio se enteren del cambio.

5.4 Mapa de archivos

Rutas relativas a la raíz del proyecto (`app/`); `infra/` abrevia `infrastructure/`.

Capa	Archivo	Responsabilidad
Dominio	`domain/models.py`	Cuatro objetos inmutables: `FileToUpload`, `StoredFile`, `FileRecord`, `FileContent`. Cero dependencias externas.
Aplicación	`application/ports.py`	Los dos puertos (`FileStoragePort`, `FileMetadataPort`) y la excepción `FileStorageError`.
Aplicación	`application/upload_service.py`	Caso de uso de subida. Valida y orquesta los dos puertos. Límite propio: 250 MB.
Aplicación	`application/file_query_service.py`	Casos de uso de lectura: listar el catálogo y obtener contenido por ID.
Infra	`infra/config.py`	Configuración tipada desde variables de entorno. Falla al arrancar si falta algo.
Infra	`infra/graph/auth.py`	MSAL *app-only*. `get_token()` renueva el token automáticamente.
Infra	`infra/graph/client.py`	Cliente Graph compartido: Bearer token, traducción de errores HTTP, caché del `site_id`.
Infra	`infra/graph/sharepoint_adapter.py`	Biblioteca: subida simple y por trozos, descarga por ID, listado.
Infra	`infra/graph/sharepoint_list_adapter.py`	Lista: inserta y lee filas de metadata.
Infra	`infra/http/upload_controller.py`	Router de la API. Traduce HTTP a dominio y mapea excepciones a códigos.
Raíz	`main.py`	*Composition root*: el único lugar donde se eligen las implementaciones concretas.
Demo	`public/index.html`	Frontend de dos pestañas. No reutilizable, solo para probar la API.

5.5 Detalles técnicos de Microsoft Graph

Autenticación

Flujo *client credentials* (app-only): la aplicación se autentica con su propio `client_id` + `client_secret`, sin usuario logueado. Aplican los permisos de tipo Application del registro en Entra ID, no los delegados. El `scope` siempre es `https://graph.microsoft.com/.default`.

El token se renueva solo: la función de autenticación se invoca antes de cada petición y devuelve el token cacheado si sigue vigente (1 hora), pidiendo uno nuevo cuando expira. No hay que renovarlo a mano ni reiniciar la aplicación.

No confundir

El token de acceso (dura 1 hora, se renueva solo) es distinto del `client_secret` (dura meses o años, según lo elegido al crearlo en Azure). Cuando el `client_secret` caduca, la autenticación falla y hay que generar uno nuevo en el portal.

Subida: dos mecanismos según el tamaño

Tamaño	Mecanismo	Endpoint
$\leq$ 4 MB	PUT directo	`PUT /drives/{drive-id}/root:/{ruta}:/content`

$>$ 4 MB	*Upload session* por trozos	`POST .../createUploadSession` + `PUT` con `Content-Range`

El límite de 4 MB y la exigencia de que los trozos sean múltiplos de 320 KiB son reglas de Microsoft Graph, no del proyecto. El template usa trozos de 3.2 MB (10 × 320 KiB). La `uploadUrl` que devuelve la sesión viene pre-autorizada: esos `PUT` no llevan el encabezado `Authorization`.

Lectura por identificador

La descarga usa dos llamadas: `GET /drives/{drive-id}/items/{id}` para los metadatos (nombre y MIME real), y `GET .../items/{id}/content` para los bytes. La segunda responde con un `302` hacia una URL temporal de descarga.

La lista de metadatos

Insertar una fila es `POST /sites/{site-id}/lists/{list-id}/items` con los valores dentro de `{"fields": {...}}`, usando los nombres internos de las columnas. Para leer, el parámetro `?expand=fields` es obligatorio: sin él, Graph devuelve los metadatos del elemento pero no los valores de las columnas.

La fecha de inserción no necesita columna propia: toda lista de SharePoint trae las columnas automáticas `Created` y `Modified`, que Graph expone como `createdDateTime`. Al generarlas el servidor, son más confiables que una fecha enviada por el cliente.

5.6 Trampas encontradas en la implementación real

Estos problemas se diagnosticaron contra el tenant real y no son obvios desde la documentación de Microsoft.

Una columna "Hipervínculo" rompe la inserción

Una columna de metadata creada como *Hipervínculo* en vez de *texto* hace que Graph responda `500 generalException` — un mensaje que no nombra la columna culpable. **Solución:** crear todas las columnas como "Una línea de texto" (salvo el tamaño, que va como "Número"). Si ya existe mal creada, hay que recrearla desde la interfaz de SharePoint: borrar columnas por API puede devolver `403` aunque la aplicación tenga permisos de escritura.

Los nombres internos de columna no son los que se ven en pantalla

SharePoint deriva el nombre interno del nombre con que se creó la columna, y lo codifica si tenía espacios o tildes (por ejemplo, "Tamaño (bytes)" queda con caracteres codificados). Por eso los nombres internos se configuran por variables de entorno (`LIST_COL_*`) y no quedan fijos en el código. Se verifican con `GET /sites/{site-id}/lists/{list-id}/columns`.

Una excepción no controlada devuelve HTML y rompe el frontend

Un fallo de autenticación no capturado hacía que el servidor respondiera con una página HTML de error en vez de JSON, y el navegador fallaba con un mensaje de parseo que ocultaba la causa real. **Solución:** los errores de autenticación se traducen a la excepción propia del dominio, y el frontend tolera respuestas que no son JSON.

Otros errores frecuentes

- `401 invalid_client` — secret vencido o mal copiado (se copia el *Value*, no el *Secret ID*).

- `403 Access denied` — falta el permiso de aplicación o el admin consent.
- `404` al resolver el sitio — el hostname o la ruta no coinciden con la URL real del sitio.
- `400 Field 'X' is not recognized` — nombres internos de columna mal configurados.

5.7 Endpoints expuestos por el template

Método	Ruta	Comportamiento
POST	`/api/files`	Sube un archivo (multipart, campo `file`). Devuelve `id`, `name`, `size`, `webUrl` y el registro de metadata insertado.
GET	`/api/files`	Lista el catálogo desde la lista de metadatos, más reciente primero.
GET	`/api/files/{id}/content`	Devuelve los bytes con el MIME real. Usable como `src` de una imagen.
GET	`/api/health`	Vida del servicio. No consulta Microsoft Graph.
GET	`/docs`	Documentación interactiva generada automáticamente.

5.8 Estado verificado y limitaciones conocidas

Se verificó contra el tenant real (sitio `/sites/app-crud`, biblioteca "ARCHIVOS DE PRUEBA", lista "DATA FILES"): autenticación correcta, subida de un archivo de varios megabytes a la biblioteca, fila insertada en la lista con todas sus columnas pobladas y `DriveItemId` coincidiendo entre ambas, y lectura correcta del catálogo desde la interfaz.

Al ser una prueba de concepto, tiene limitaciones deliberadas, a tener en cuenta antes de llevarlo a producción:

- **Sin transaccionalidad.** Si el archivo sube pero la inserción en la lista falla, la petición falla completa y el archivo queda en la biblioteca sin su fila.
- **El archivo se carga completo en memoria** antes de subirlo; para archivos muy grandes conviene procesar por trozos.
- **Sin autenticación de usuario** propia de la aplicación.
- **Sin paginación** en el listado del catálogo.
- **Sin reintentos ni manejo de límite de tasa** (Graph responde `429` bajo carga).
- El caché del token es por proceso: con varias instancias, cada una mantiene el suyo.
- **Sin validación de tipo de archivo** ni de contenido malicioso.

5.9 Cómo adoptarlo en otro proyecto

La pieza reutilizable es la carpeta de infraestructura de Graph junto con los puertos que la abstraen. No depende del framework web usado: funciona igual desde otro framework de Python o desde un proceso en segundo plano.

1. Copiar los puertos, la autenticación, el cliente y el adaptador de biblioteca. El adaptador de lista solo si se va a usar una lista de SharePoint como catálogo; con base de datos propia, se implementa el puerto de metadata contra la tabla correspondiente.
2. Copiar los modelos de dominio o mapearlos a las entidades propias: el adaptador solo necesita nombre, bytes y tipo MIME.
3. Agregar las dependencias de autenticación MSAL y de peticiones HTTP.

4. Cada aplicativo debe tener su propio registro en Entra ID y su propio secret, no compartir el de esta prueba de concepto.
5. En los servicios, depender del puerto y no del adaptador; ensamblar las implementaciones concretas en el punto de arranque de la aplicación.

6 Resumen del proceso

En conjunto, el proceso documentado cubre cuatro frentes complementarios: la identidad y permisos de la aplicación en Microsoft Entra ID, la estructura del sitio de SharePoint que recibe la información, la validación manual de cada llamada en Postman, y el template funcional que finalmente automatiza todo el flujo en producción.

El punto que con mayor frecuencia genera confusión al replicar este proceso es el paso 8: el permiso de acceso a sitios habilita el tipo de acceso a nivel de Microsoft Graph, pero requiere una autorización adicional, sitio por sitio, para que la aplicación pueda realmente leer o escribir en él. El segundo punto de mayor fricción, ya dentro del template, es el tipo de columna en la lista de metadatos: debe ser texto plano, nunca hipervínculo.